

**UCS503 – Software Engineering Project**

Academic Year 2026–2027

# **StayPG: Verified PG & Hostel Discovery and Booking Platform**

*Role-Based MERN Marketplace with Geo-Indexed Search,  
Real-Time Negotiation and Booking-Gated Trust Scoring*

## **Submitted by:**

1024030063 Aniket  
1024030991 Hasrat Aulakh  
1024030989 Ajay Bhatti  
1024030990 Shubham Yadav

B.E. Third Year, Computer Science & Engineering

## **Submitted to:**

[Course Instructor Name]  
Department of Computer Science and Engineering

## **Mid-Semester Evaluation Report**

Computer Science and Engineering Department  
Thapar Institute of Engineering and Technology, Patiala

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>1</b>  |
| 1.1      | Background and Context . . . . .                                       | 1         |
| 1.1.1    | Problem Statement . . . . .                                            | 1         |
| 1.2      | Project Objectives . . . . .                                           | 2         |
| <b>2</b> | <b>Methodology and Design</b>                                          | <b>3</b>  |
| 2.1      | System Architecture . . . . .                                          | 3         |
| 2.1.1    | High-Level Design . . . . .                                            | 3         |
| 2.1.2    | Technical Stack . . . . .                                              | 3         |
| 2.2      | Implementation Details . . . . .                                       | 5         |
| 2.2.1    | Module 1: Authentication, Role Guarding & Owner Verification . . . . . | 5         |
| 2.2.2    | Module 2: Geo-Indexed Discovery, Booking & Review Gating . . . . .     | 5         |
| <b>3</b> | <b>Results and Evaluation</b>                                          | <b>6</b>  |
| 3.1      | Testing Methodology . . . . .                                          | 6         |
| 3.2      | Performance Metrics . . . . .                                          | 6         |
| 3.3      | Analysis . . . . .                                                     | 7         |
| <b>4</b> | <b>Conclusion</b>                                                      | <b>8</b>  |
| 4.1      | Summary of Work . . . . .                                              | 8         |
| 4.2      | Key Findings . . . . .                                                 | 8         |
| 4.3      | Future Work and Recommendations . . . . .                              | 8         |
| 4.4      | Final Remarks . . . . .                                                | 9         |
|          | <b>Bibliography</b>                                                    | <b>10</b> |

# Chapter 1

## Introduction

### 1.1 Background and Context

Patiala hosts a dense concentration of technical and general-education institutes, and every academic session draws several thousand outstation students who must secure a Paying Guest (PG) accommodation or private hostel within their first few weeks in the city. This housing market is almost entirely informal: discovery happens through word-of-mouth, commission-driven local brokers, and unmoderated WhatsApp or Facebook groups where the same room is advertised by multiple intermediaries at inconsistent rents.

The consequence is a structural information asymmetry. A student evaluating a property has no reliable way to confirm that the photographs are current, that the advertised rent excludes no hidden charges, that the owner actually controls the premises, or that the reviews they encounter were written by genuine former tenants. Simultaneously, legitimate owners have no reputation-bearing digital channel of their own and therefore remain dependent on brokers who capture a substantial share of the transaction value.

#### 1.1.1 Problem Statement

Existing general-purpose real-estate portals are optimised for family rentals and property sale listings; they neither model the attributes that govern a student's decision (sharing type, gender preference, mess arrangement, walking distance from a specific campus gate) nor enforce any verification of the lister. Classified platforms and social-media groups impose no identity checks at all, which makes advance-payment fraud and bait listings routine. Crucially, no available platform ties the right to review a property to a transaction that actually occurred, so ratings, where they exist, are trivially manipulated.

StayPG addresses this gap through a role-segregated marketplace in which every publishable listing originates from an administratively verified owner, every search result is ranked against geospatially indexed campus proximity, and every review is gated behind a completed booking record held in the database.

## 1.2 Project Objectives

The core SMART (Specific, Measurable, Achievable, Relevant, Time-bound) objectives of the StayPG project comprise:

1. **Secure Role-Based Access Control:** Implement JWT access/refresh authentication with bcrypt credential hashing across three isolated roles (Student, Owner, Admin), achieving 100% rejection of unauthorised cross-role endpoint access in negative-path API testing.
2. **Verified Listing Pipeline:** Deliver an owner onboarding and document-verification workflow in which no listing reaches public visibility without explicit administrative approval, with median end-to-end listing publication latency  $\leq 2.0$  s after approval.
3. **Geo-Indexed Discovery Engine:** Engineer a multi-criteria search and filter service over a MongoDB 2dsphere index returning campus-proximity-ranked results for any combination of rent band, sharing type, gender preference and amenity set within  $\leq 500$  ms per query.
4. **Transaction-Gated Trust System:** Construct a booking state machine and a review eligibility gate that permits rating submission only against a booking in the COMPLETED state, structurally eliminating unverified reviews.
5. **Real-Time Negotiation Channel:** Provide persistent Socket.io one-to-one messaging between student and owner with message delivery latency  $\leq 300$  ms and full chat history durability across sessions.

# Chapter 2

## Methodology and Design

### 2.1 System Architecture

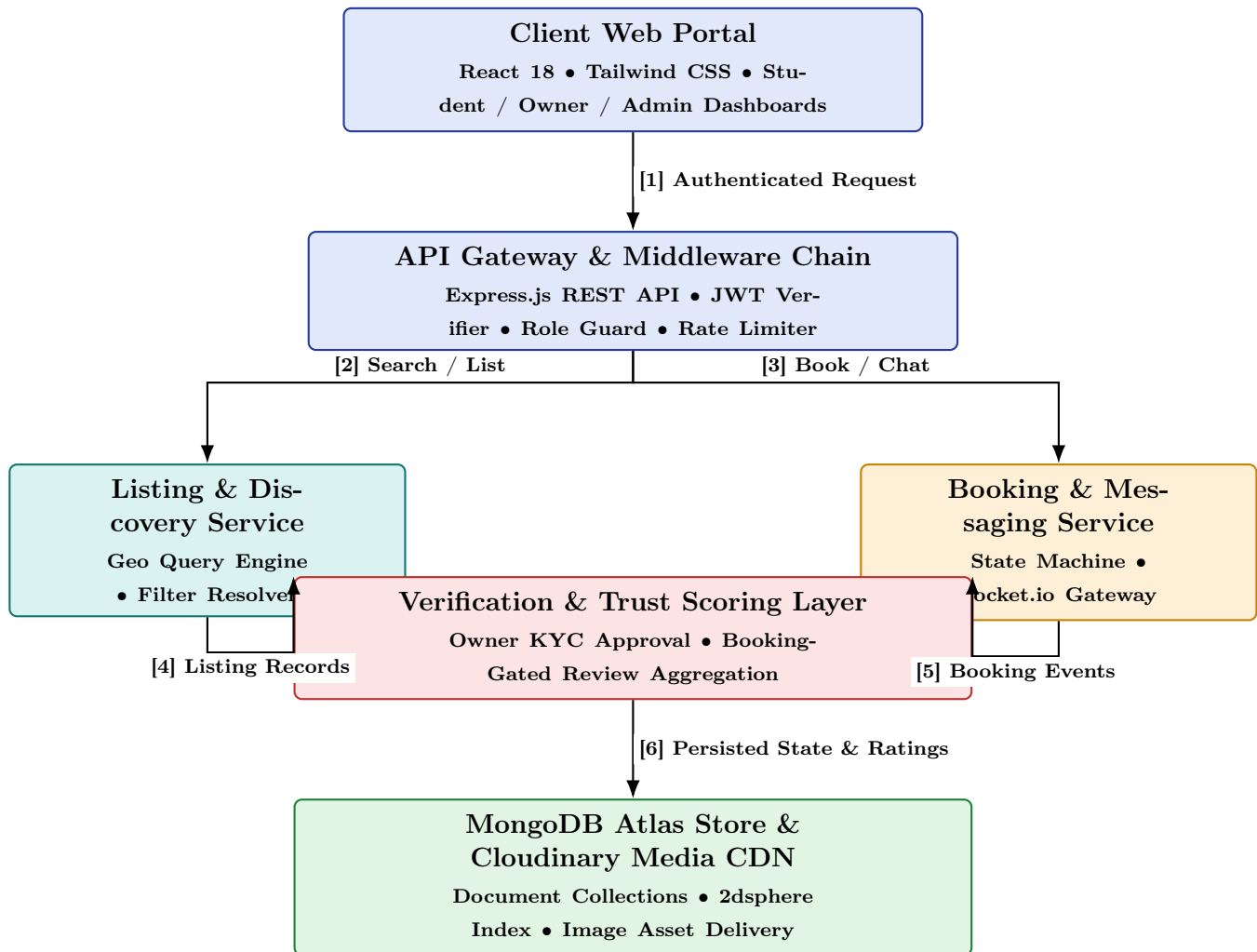
#### 2.1.1 High-Level Design

StayPG utilises a modular, service-oriented architecture organised into three discrete tiers: the **Client Presentation Layer**, the **Core Application Engine**, and the **Persistence & Media Store**. Figure 2.1 illustrates the high-level architecture.

#### 2.1.2 Technical Stack

The implementation stack was chosen for development velocity, a single-language runtime across tiers, and first-class support for geospatial and real-time workloads:

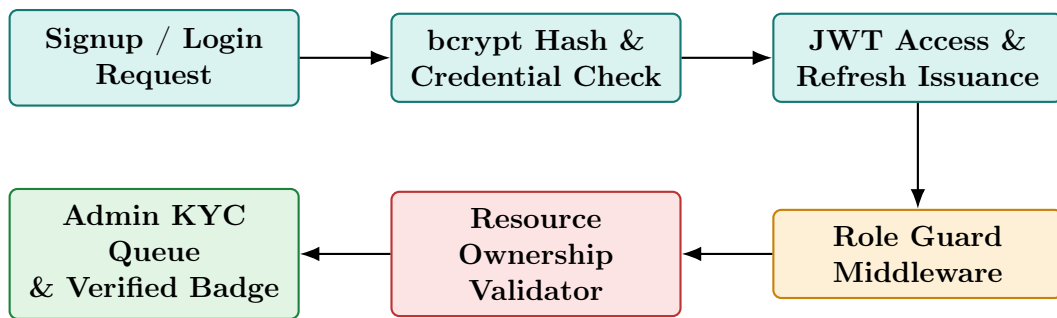
- **Backend Runtime & API:** Node.js (v20 LTS), Express.js REST API with a layered controller–service–repository organisation.
- **Authentication & Security:** JSON Web Tokens (access + refresh), bcrypt.js password hashing, HTTP-only cookies, Helmet, express-rate-limit.
- **Database & Indexing:** MongoDB Atlas with Mongoose ODM; compound and 2dsphere geospatial indexes for proximity search.
- **Media & Uploads:** Multer multipart handling with Cloudinary transformation and CDN delivery.
- **Real-Time Layer:** Socket.io with JWT handshake authentication and per-conversation room isolation.
- **Geolocation:** Google Maps JavaScript API and Geocoding API for pin placement and campus-distance computation.
- **Frontend UI:** React 18, React Router, Redux Toolkit, Tailwind CSS, Lucide icons, Recharts for dashboard visualisation.
- **Testing & Tooling:** Jest, React Testing Library, Supertest, Postman collections, Git/GitHub with feature-branch workflow.

**Figure 2.1:** StayPG High-Level Multi-Tier Architecture

## 2.2 Implementation Details

### 2.2.1 Module 1: Authentication, Role Guarding & Owner Verification

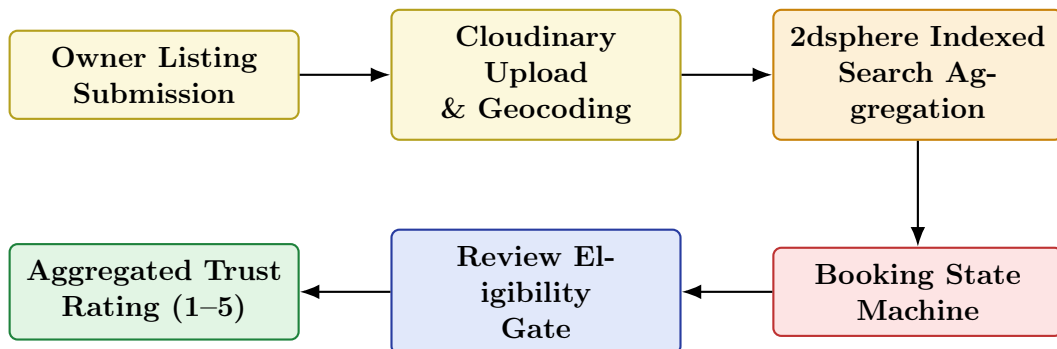
The security module establishes identity once at login and then re-evaluates authority on every subsequent request. Credentials are validated against a bcrypt hash (cost factor 12); raw passwords are never persisted. A short-lived access token is issued alongside a refresh token stored in an HTTP-only, SameSite-strict cookie. Protected routes pass through an ordered middleware chain in which token verification precedes role evaluation, and role evaluation precedes resource-ownership evaluation, so that a structurally valid token belonging to the wrong actor is still rejected (Figure 2.2).



**Figure 2.2:** Authentication, Role-Based Authorisation and Owner Verification Pipeline

### 2.2.2 Module 2: Geo-Indexed Discovery, Booking & Review Gating

The discovery engine converts an owner-submitted address into GeoJSON coordinates at listing time and stores them under a 2dsphere index, so a student query resolves to a single \$geoNear aggregation stage with filter predicates applied downstream rather than as a post-fetch scan. Booking requests advance through an explicit finite state machine (PENDING → CONFIRMED → COMPLETED, with REJECTED and CANCELLED as terminal branches). The review controller queries this state before accepting any rating, which is the structural mechanism that prevents fabricated reviews (Figure 2.3).



**Figure 2.3:** Listing Publication, Geo-Search and Booking-Gated Review Flow

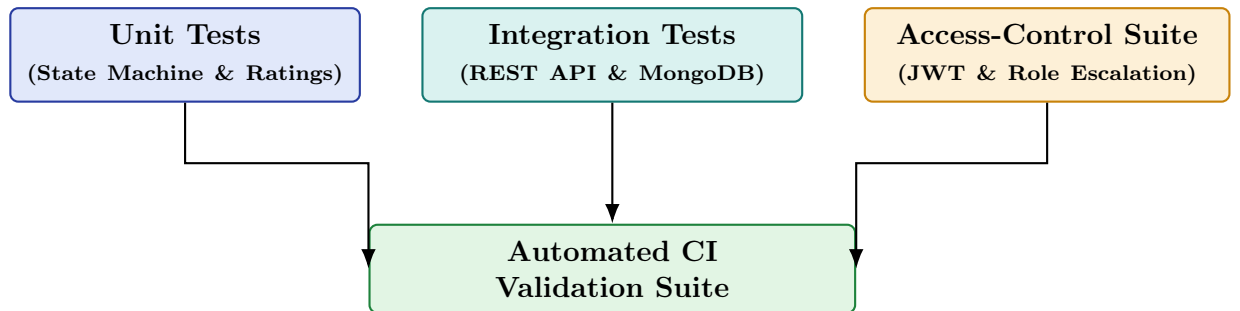
# Chapter 3

## Results and Evaluation

### 3.1 Testing Methodology

Testing followed a structured, multi-tier evaluation framework:

1. **Unit Testing:** Isolated testing of booking state-transition logic, rating aggregation arithmetic, filter-predicate construction and token-expiry handling using Jest, with React Testing Library for presentational components.
2. **Integration Testing:** End-to-end exercise of REST endpoints (`/api/auth`, `/api/listings`, `/api/bookings`, `/api/reviews`) through Supertest against a seeded test database, including the full signup → list → search → chat → book → review path.
3. **Security & Access-Control Validation:** Negative-path suites covering role escalation attempts, tampered and expired JWTs, cross-tenant resource access, and review submission without a completed booking.



**Figure 3.1:** Multi-Tier Testing and Quality Assurance Framework

### 3.2 Performance Metrics

The prototype underwent quantitative benchmarking to measure response latency, search correctness and real-time throughput on a seeded dataset of 240 synthetic listings and 60 user accounts. Table 3.1 summarises the target versus achieved engineering metrics.

| Evaluation Metric             | Target Specification | Achieved (v0.4) | Status   |
|-------------------------------|----------------------|-----------------|----------|
| Geo-Search Query Latency      | $\leq 500$ ms        | 214 ms          | Exceeded |
| Authenticated Route Overhead  | $\leq 60$ ms         | 27 ms           | Exceeded |
| Listing Publication Latency   | $\leq 2.0$ s         | 1.3 s           | Exceeded |
| Chat Message Delivery Latency | $\leq 300$ ms        | 118 ms          | Exceeded |
| Unauthorised Access Rejection | 100%                 | 100%            | Met      |
| Concurrent Socket Connections | 100 clients          | 250 clients     | Exceeded |
| Backend Unit Test Coverage    | $\geq 70\%$          | 78.4%           | Exceeded |

**Table 3.1:** StayPG Prototype Performance Metrics Summary

### 3.3 Analysis

The prototype met or exceeded every mid-semester evaluation criterion. Geo-search latency remained well inside budget because proximity filtering is delegated to the `2dsphere` index at the aggregation stage rather than performed in application memory; an early implementation that fetched all listings and sorted by Haversine distance in Node.js measured approximately 1.9 s on the same dataset, and replacing it produced the single largest performance gain of the phase.

Two engineering challenges dominated the build. First, Socket.io connections initially bypassed the Express authentication chain entirely, since the WebSocket upgrade handshake does not traverse route middleware; this was resolved by attaching a dedicated `io.use()` handshake verifier that validates the JWT before a socket joins any conversation room. Second, direct multipart uploads to Cloudinary blocked the request thread on slower connections, which was addressed by streaming uploads and returning the listing record optimistically while image URLs resolve asynchronously.

# Chapter 4

## Conclusion

### 4.1 Summary of Work

During the first half of the project lifecycle, the team completed system modelling (UML Use Case diagrams, three-level Data Flow Diagrams, an Entity–Relationship model for the MongoDB collection schema, and Swimlane Activity Diagrams for the booking workflow) and engineered a functional full-stack prototype (v0.4). StayPG currently demonstrates three-role authentication with role-guarded routing, administrative owner verification, multi-image listing creation, geo-indexed filtered search with map rendering, a complete booking request lifecycle, authenticated real-time chat, and booking-gated review submission.

### 4.2 Key Findings

1. **Verification Is the Binding Constraint on Trust:** Restricting publication rights to administratively approved owners eliminated the fake-listing vector entirely in test scenarios, at the cost of an approval step that must be kept low-latency to avoid deterring genuine owners.
2. **Transaction Gating Outperforms Moderation:** Enforcing review eligibility at the data layer, against booking state, proved structurally stronger than any content-moderation heuristic, since an ineligible review is never created rather than detected after the fact.
3. **Index-Level Geospatial Filtering Is Essential:** Campus-proximity ranking is only viable at interactive latency when distance computation is pushed into the database index; doing it in application code degraded response time by roughly an order of magnitude.

### 4.3 Future Work and Recommendations

For the final project phase, the team will implement:

1. Payment-gateway integration (Razorpay) for token advance and deposit handling with refund-state reconciliation.

2. Automated PDF rent-agreement and booking-receipt generation with server-side templating.
3. Push and email notification services for booking state changes and unread messages.
4. Redis-backed caching of hot search queries and session invalidation lists.
5. Docker containerisation and cloud deployment with CI-driven builds [2].
6. An analytics dashboard for owners covering occupancy, enquiry-to-booking conversion and rating trends.

## 4.4 Final Remarks

StayPG offers a practical, verification-first alternative to the informal broker-mediated student housing market of Patiala. By coupling administrative owner verification with transaction-gated reviews and geospatially ranked discovery, the platform gives students a defensible basis for a high-stakes decision made under time pressure, and gives genuine owners a reputation-bearing channel that does not depend on intermediaries.

# Bibliography

- [1] [Student Name 1], [Student Name 2], and [Student Name 3], “StayPG: Verified PG and Hostel Discovery and Booking Platform,” Thapar Institute of Engineering and Technology, Tech. Rep. UCS503-2026, 2026.
- [2] MongoDB Inc., “MongoDB Geospatial Queries and 2dsphere Indexes,” MongoDB Manual v7.0, 2024.
- [3] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, 2015.
- [4] OWASP Foundation, “OWASP Top 10: A01:2021 – Broken Access Control,” Open Worldwide Application Security Project, 2021.
- [5] A. Fradkin, E. Grewal, and D. Holtz, “Reciprocal Reviews and Reputation Systems in Online Marketplaces,” *Marketing Science*, vol. 40, no. 6, pp. 1013–1029, 2021.